PROYECTO 1

Diseño e implementación

Escrito por: Camila Correa González (202320669), Samuel Pulido Morón (202420861), Natalia Salazar Tena (202421753)

Presentación del proyecto

Nuestro proyecto consiste en el diseño e implementación de un sistema para la gestión de un Board Game Café, el cual integra funcionalidades clave del negocio como la administración de usuarios, préstamos de juegos, ventas de productos y gestión operativa del café.

El sistema está centrado en la clase **BoardgameCafe**, que actúa como núcleo y permite coordinar todas las operaciones principales. A partir de allí, se modelaron diferentes componentes como usuarios (clientes, empleados y administrador), juegos de mesa, ventas, turnos y sugerencias, cada uno con responsabilidades claras y separadas.

Soporte a Funcionalidades:

Despueés de que corran el código podrán darse cuenta de que estos son los componentes con respectivas responsabilidades; los mencionamos para que puedan compararlos con el código en el compilador de Eclipse: El diseño soporta de forma clara y organizada las funcionalidades del sistema mediante una adecuada división en componentes. La clase **BoardgameCafe** actúa como núcleo, coordinando las operaciones principales como registro de usuarios, asignación de mesas, préstamos, ventas, turnos e informes.

El componente de **usuarios** (Cliente, Empleado, Administrador, Mesero, Cocinero) distribuye las responsabilidades según el rol, permitiendo manejar correctamente las acciones específicas de cada tipo de usuario.

El componente de **juegos y préstamos** (JuegoDeMesa, Prestamo, PuedePrestar) se encarga del control de inventario y de las reglas de alquiler, mientras que el componente de **ventas** (VentaCafe, VentaJuego y sus detalles) maneja la venta de alimentos y juegos, incluyendo cálculos de totales, impuestos y descuentos.

Adicionalmente, el componente de **gestión operativa** (Mesa, Turno, SolicitudCambioTurno, SugerenciaPlatillo) permite administrar la ocupación del café y la organización del personal.

Finalmente, el componente de **persistencia** separa el almacenamiento del resto del sistema, manteniendo un bajo acoplamiento.

En conjunto, el diseño cubre las funcionalidades requeridas para todos los tipos de usuario y distribuye las responsabilidades de manera clara, coherente y organizada.

Documentación de Persistencia:

La persistencia del sistema se maneja mediante una arquitectura desacoplada compuesta por la interfaz **IPersistenciaBoardGameCafe**, la clase intermediaria **CentralPersistencia** y una implementación concreta basada en archivos de texto. Esta estructura permite separar la lógica de almacenamiento de la lógica del negocio, facilitando el mantenimiento y la posible extensión del sistema.

La clase de persistencia se encarga de **guardar y cargar el estado completo del sistema** (usuarios, mesas, juegos, préstamos, ventas, turnos, etc.) en un archivo externo ubicado en la ruta src/datos/. De esta forma, se garantiza que la información no se pierda entre ejecuciones del programa.

El proceso de guardado consiste en recorrer los componentes principales del sistema y escribir su información en el archivo, mientras que el proceso de carga reconstruye los objetos y restablece sus relaciones, asegurando que el estado del café se recupere de manera consistente.

Esta solución es coherente con el enunciado del problema, ya que cumple con el requerimiento de manejar persistencia de la información en archivos y permite mantener el estado del negocio de forma clara, estructurada y reutilizable.

Justificación de Decisiones de Diseño:

Durante el desarrollo del sistema se tomaron decisiones enfocadas en representar correctamente el funcionamiento de un Board Game Café, manteniendo un diseño organizado, coherente y fácil de extender.

1. Uso de una clase central BoardgameCafe como núcleo del sistema

Se decidió que BoardgameCafe fuera la clase que concentra el estado principal del sistema y coordina la mayor parte de la lógica de negocio. Esta clase mantiene referencias al administrador, clientes, empleados, mesas, juegos, préstamos, productos del menú, ventas, turnos, solicitudes y sugerencias. Además, implementa directamente operaciones importantes como crear préstamos, registrar ventas, recibir clientes, aprobar sugerencias y gestionar cambios de turno.

La justificación de esta decisión es que el café funciona como una **unidad de negocio centralizada**: todas las entidades dependen del estado global del establecimiento y muchas operaciones requieren validar reglas cruzadas entre varias clases. Por ejemplo, para prestar un juego se necesita revisar quién lo solicita, si tiene mesa, si hay menores, si el juego está disponible y si existe un mesero capacitado en caso de que el juego sea difícil. Esa coordinación se expresa claramente dentro de BoardgameCafe.

La principal implicación es positiva en términos de claridad y control del flujo: la lógica importante queda localizada en un solo punto. Como contrapartida, esta clase puede crecer bastante, por lo que en versiones futuras podría separarse en servicios especializados si se quisiera reducir tamaño o aumentar modularidad.

2. Separación entre entidades del dominio y lógica de coordinación

Aunque BoardgameCafe concentra la coordinación, las clases del dominio conservan responsabilidades propias. Por ejemplo, Mesa administra su ocupación y capacidad; JuegoDeMesa controla disponibilidad, préstamo, devolución y copias de venta; VentaCafe calcula subtotal, impuesto y propina; VentaJuego calcula subtotal, IVA y total; Cliente maneja puntos, mesa actual y juegos prestados.

Esto se hizo para respetar el principio de **encapsulamiento**: cada clase conoce y protege su propio estado. No tendría sentido que, por ejemplo, BoardgameCafe modificara directamente contadores internos de un juego o recalculara manualmente el total de una venta cuando esas reglas pertenecen naturalmente a las entidades involucradas.

La implicación es que el sistema queda más ordenado y menos propenso a inconsistencias, porque cada objeto controla sus propias reglas básicas.

3. Uso de herencia para modelar usuarios y productos

Se definió una clase abstracta Usuario con atributos comunes como id, nombre, login y contraseña, y a partir de ella se especializaron Administrador, Cliente y Empleado. A su vez, Empleado es abstracta y sirve de base para Mesero y Cocinero.

La razón de esta decisión es que estos tipos comparten información y comportamiento general, pero cada uno tiene restricciones y responsabilidades distintas. El administrador no pide préstamos ni realiza compras; el cliente puede tener mesa, puntos de fidelidad y código de descuento; el empleado tiene turnos, estado de trabajo y eventualmente descuento o conocimientos particulares. La herencia evita duplicación y hace explícitas las diferencias del dominio.

Del mismo modo, ProductoMenu es abstracta y se especializa en Bebida y Pasteleria, ya que ambos son productos del menú, pero solo las bebidas tienen propiedades como alcohólica o caliente, mientras que la pastelería maneja alérgenos.

La implicación principal es una mejor **reutilización y legibilidad del modelo**, además de permitir tratar diferentes subtipos de forma polimórfica cuando conviene.

4. Uso de clases abstractas cuando no tiene sentido instanciar el concepto general

Usuario, Empleado, ProductoMenu y Venta fueron definidas como clases abstractas.

La justificación es conceptual y de diseño. No tiene sentido crear un “usuario genérico” sin rol, ni un “empleado genérico” sin función concreta, ni una “venta genérica” sin detallar qué se vendió. Definirlas como abstractas evita usos incorrectos del modelo y obliga a instanciar tipos válidos del dominio.

La implicación es que se refuerza la consistencia del sistema y se reduce la posibilidad de errores de modelado.

5. Uso de la interfaz PuedePrestar para unificar el comportamiento de préstamo

Se creó la interfaz PuedePrestar con operaciones para verificar si alguien puede pedir un préstamo, agregar o quitar juegos prestados y consultar los juegos prestados. Esta interfaz es implementada tanto por Cliente como por Empleado.

La decisión se tomó porque, aunque clientes y empleados son diferentes, ambos pueden pedir juegos prestados bajo ciertas condiciones. En lugar de duplicar lógica o depender de verificaciones rígidas por tipo, se abstrajo el comportamiento común en una interfaz.

Esto tiene una implicación muy importante: BoardgameCafe puede trabajar con cualquier objeto que cumpla el contrato PuedePrestar, lo que reduce acoplamiento y mejora extensibilidad. Si en el futuro apareciera otro rol autorizado para pedir préstamos, bastaría con implementar la interfaz.

6. Uso de composición para representar ventas con detalles

Las ventas no guardan únicamente un total, sino una colección de detalles: VentaCafe contiene DetalleVentaCafe y VentaJuego contiene DetalleVentaJuego. Cada detalle representa una línea individual de la venta, con producto o juego, cantidad y subtotal.

La justificación es que una venta real puede involucrar varios ítems, y el sistema necesita registrar esa desagregación para fines contables, de auditoría y de cálculo. Esta decisión hace posible calcular subtotales, impuestos, descuentos y propinas de forma transparente y coherente con el dominio.

La implicación es que el modelo representa mejor una venta real y deja abierta la posibilidad de generar informes más detallados más adelante.

7. Manejo explícito de reglas de negocio complejas dentro del dominio

Una decisión crítica fue modelar reglas importantes con validaciones explícitas en los métodos del sistema. Por ejemplo:

- un cliente no puede pedir préstamo si no está en el café o si ya tiene dos juegos;
- un juego no se presta si no hay copias disponibles;
- no se puede prestar un juego no apto para la mesa por número de jugadores o por edad;
- si un juego es difícil, debe existir un mesero en turno que lo domine;
- no se pueden servir bebidas alcohólicas a mesas con menores ni bebidas calientes si la mesa tiene un juego de acción;
- un cambio de turno solo se aprueba si se mantiene el personal mínimo requerido.

La justificación es que estas reglas no son accesorias: son parte del corazón funcional del sistema y por eso deben estar implementadas en el modelo y no solo en la interfaz o en pruebas externas.

La implicación es que el sistema protege mejor la integridad del negocio y garantiza que cualquier presentación futura respete las mismas restricciones.

8. Modelado de turnos y solicitudes

Se usaron clases como Turno y SolicitudCambioTurno para representar estos procesos.
Esto permite manejar validaciones y estados de forma clara y extensible.

9. Uso de estructuras de datos adecuadas

Se usan HashMap para búsquedas rápidas (clientes, empleados) y listas para colecciones recorribles (juegos, ventas, mesas).
Esto mejora eficiencia y organización del sistema.

10. Desacoplamiento de la persistencia

Se utilizó una interfaz (IPersistencia) y una clase intermedia (CentralPersistencia).
Esto permite cambiar la forma de guardar datos sin afectar la lógica del sistema.

11. Persistencia en archivos externos al código fuente

El enunciado exige que la información sea persistente y almacenada en archivos. La solución adoptada cumple esa restricción mediante una clase especializada de persistencia y una ruta separada del código fuente, lo cual es consistente con el requerimiento técnico del proyecto.

La justificación es directa: el sistema debe poder reconstruir el estado del café entre ejecuciones, y por ello era necesario incluir una capa de almacenamiento.

La implicación principal es que el sistema deja de depender únicamente de la memoria temporal del programa y se vuelve utilizable en escenarios reales de prueba.

Conclusión

El diseño se basa en encapsulamiento, bajo acoplamiento y representación fiel del dominio.
Las decisiones tomadas permiten un sistema claro, mantenible y alineado con los requerimientos del proyecto.

Asignación de Responsabilidades:

El diseño del sistema presenta una **distribución coherente y sobretodo directa de responsabilidades**, donde cada componente cumple un rol específico dentro del funcionamiento del Board Game Café, evitando la sobrecarga de funciones en una sola clase y facilitando el mantenimiento y la comprensión del sistema.

La clase **BoardgameCafe** actúa como el componente central y tiene la responsabilidad de **gestionar y coordinar la lógica del negocio**, manteniendo el estado global del sistema. Desde esta clase se controlan las operaciones principales como el registro de usuarios, la asignación de mesas, la creación de préstamos, el registro de ventas, la gestión de turnos, la aprobación de solicitudes y la generación de informes. Su responsabilidad es orquestar el funcionamiento del sistema, no reemplazar la lógica interna de las demás clases.

El componente de **usuarios** está conformado por Usuario, Cliente, Empleado, Administrador, Mesero y Cocinero. Cada uno tiene responsabilidades diferenciadas: el cliente administra su participación en el sistema (mesa, puntos, préstamos y compras), el empleado gestiona su disponibilidad y turnos, y el administrador supervisa y controla procesos del sistema. La especialización en mesero y cocinero permite modelar funciones específicas, como el conocimiento de juegos en el caso del mesero.

El componente de **juegos y préstamos** (JuegoDeMesa, Prestamo, PuedePrestar) se encarga de la gestión del inventario y de las reglas asociadas al alquiler. JuegoDeMesa controla la disponibilidad, copias y restricciones del juego; Prestamo representa cada transacción de préstamo; y la interfaz PuedePrestar define un comportamiento común para quienes pueden solicitar juegos. Esto permite manejar de forma clara y consistente las reglas del negocio relacionadas con el uso de juegos.

El componente de **ventas** (Venta, VentaCafe, VentaJuego, DetalleVentaCafe, DetalleVentaJuego) tiene la responsabilidad de representar y calcular las transacciones del sistema. Se separa la venta en detalles para permitir manejar múltiples productos por operación y calcular correctamente subtotales, impuestos, descuentos y totales, lo que facilita la generación de reportes financieros.

El componente de **gestión operativa** (Mesa, Turno, SolicitudCambioTurno, SugerenciaPlatillo) administra aspectos clave del funcionamiento diario del café, como la ocupación de mesas, la organización del personal, los cambios de turno y la incorporación de nuevos productos. Estas clases encapsulan reglas específicas que no pertenecen directamente a ventas o préstamos, pero son esenciales para el negocio.

Finalmente, el componente de **persistencia** (IPersistenciaBoardGameCafe, CentralPersistencia y su implementación) tiene la responsabilidad de **almacenar y recuperar el estado del sistema** en archivos externos. Esta responsabilidad está separada de la lógica del dominio, lo que permite mantener un bajo acoplamiento y facilita cambios futuros en la forma de almacenamiento.

En conjunto, el diseño presenta una asignación de responsabilidades **bien definida, balanceada y alineada con los requerimientos del sistema**, donde cada clase tiene un propósito claro y contribuye de manera específica al funcionamiento general de la aplicación.